Use any other method and give full code

1 message

Pranav Madhu <pranavmadhu8055@gmail.com> Sat, 31 Oct 2026 at 11:19 am
Draft

Here is the complete code using a **different method** to parse the API response.

Instead of using `.whereType()` and `.map()`, I have replaced those sections with explicit `for` loops and `is Map<String, dynamic>` checks. This method is incredibly robust, avoids all generic type inference errors, and is perfectly safe for FlutterFlow's compiler.

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:convert';
import 'package:flutter/services.dart';
import 'package:http/http.dart' as http;
import 'package:intl/intl.dart';

class ClosureandVerificationAmbience extends StatefulWidget {
  const ClosureandVerificationAmbience({
    super.key,
    this.width,
    this.height,
    this.baseurl,
    this.branchId,
    this.empCode,
  });

  final double? width;
  final double? height;
  final String? baseurl;
  final String? branchId;
  final String? empCode;

  @override
  State<ClosureandVerificationAmbience> createState() =>
      _ClosureandVerificationAmbienceState();
}

class _ClosureandVerificationAmbienceState
    extends State<ClosureandVerificationAmbience> {
  final DateFormat _dateFormat = DateFormat('dd-MM-yyyy');

  List<_AuditOption> _auditIds = [];
  _AuditOption? _selectedAudit;
  List<_AuditRecord> _auditRecords = [];

  bool _isLoadingList = false;
  bool _isLoadingDetails = false;
  bool _isSubmitting = false;
  String? _errorMessage;

  int selectedStatus = 1; // 1: Satisfactory closed, 2: Wrong Rectification
  final _remarksController = TextEditingController();

  String get _baseUrl => (widget.baseurl ?? '').replaceAll(RegExp(r'/$'), '');

  @override
  void dispose() {
    _remarksController.dispose();
    super.dispose();
  }

  @override
  void initState() {
    super.initState();
    _loadAuditIds();
  }
```

```
Future<void> _loadAuditIds() async {
  final branchId = widget.branchId?.trim();
  if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) {
    setState(() => _errorMessage = 'Branch and API URL are required.');
    return;
  }
  setState(() {
    _isLoadingList = true;
    _errorMessage = null;
  });
  try {
    final response = await http.get(Uri.parse(
      '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
      'Audit_id_ambience/$branchId/1',
    ));
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw Exception('Unable to load Audit IDs.');
    }

    final result = _resultList(response.body);

    // Alternative Method: Using standard for-loop and explicit type checking
    final uniqueIds = <String>{};
    final List<_AuditOption> parsedAuditIds = [];

    for (final item in result) {
      if (item is Map<String, dynamic>) {
        final idStr = '${item['AUDIT_ID'] ?? ''}';
        if (idStr.isNotEmpty && uniqueIds.add(idStr)) {
          parsedAuditIds.add(_AuditOption(auditId: idStr));
        }
      }
    }

    if (!mounted) return;
    setState(() {
      _auditIds = parsedAuditIds;
      _isLoadingList = false;
    });
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isLoadingList = false;
      _errorMessage = 'Unable to load Audit IDs.';
    });
  }
}

Future<void> _loadAuditTable(_AuditOption audit) async {
  final branchId = widget.branchId?.trim();
  if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) return;
  setState(() {
    _selectedAudit = audit;
    _auditRecords = [];
    _isLoadingDetails = true;
    _errorMessage = null;
  });
  try {
    final detailsUri = Uri.parse(
      '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
      'Audit_id_Datatable/${audit.auditId}/1',
    );

    final response = await http.get(detailsUri);

    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw Exception('Unable to load audit data table.');
    }

    final detailRows = _resultList(response.body);

    // Alternative Method: Using standard for-loop and explicit type checking
    final List<_AuditRecord> parsedRecords = [];
    for (final item in detailRows) {
      if (item is Map<String, dynamic>) {
        parsedRecords.add(_AuditRecord.fromJson(item));
      }
    }

    if (!mounted) return;
    setState(() {
      _auditRecords = parsedRecords;
      _isLoadingDetails = false;
    });
```

```
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isLoadingDetails = false;
      _errorMessage = 'Unable to load datatable for the selected Audit ID.';
    });
  }
}

List<dynamic> _resultList(String body) {
  try {
    final decoded = jsonDecode(body);
    return decoded is Map<String, dynamic> && decoded['result'] is List
        ? decoded['result'] as List<dynamic>
        : <dynamic>[];
  } catch (e) {
    return <dynamic>[];
  }
}

Future<void> _showAlert(String message) async {
  if (!mounted) return;
  await showDialog<void>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Alert'),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('OK'),
        ),
      ],
    ),
  );
}

Future<void> _submitClosure(String flag) async {
  final audit = _selectedAudit;
  final branchId = widget.branchId?.trim();
  final empCode = widget.empCode?.trim();
  final remarks = _remarksController.text.trim();
  final currentDate = _dateFormat.format(DateTime.now());

  if (audit == null || branchId == null || branchId.isEmpty) {
    await _showAlert('Please select an Audit ID first.');
    return;
  }
  if (remarks.isEmpty) {
    await _showAlert('Please enter remarks.');
    return;
  }
  if (empCode == null || empCode.isEmpty) {
    setState(() => _errorMessage = 'Employee code is required.');
    return;
  }
  setState(() {
    _isSubmitting = true;
    _errorMessage = null;
  });
  try {
    final response = await http.post(
      Uri.parse('$_baseUrl/AuditInspectionApi/api/PostInseptionCtl'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'flag': 'closure_and_verify_ambience',
        'pageValue':
            'approve*$branchId~${audit.auditId}~$empCode~$selectedStatus~$remarks~$currentDate',
        'paraValue': '1',
      }),
    );
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw Exception('Unable to submit closure action.');
    }
    if (!mounted) return;
    final decodedResponse = jsonDecode(response.body);
    final responseMessage = decodedResponse is List &&
            decodedResponse.isNotEmpty &&
            decodedResponse.first is Map<String, dynamic>
        ? '${(decodedResponse.first as Map<String, dynamic>)['RES'] ?? ''}'
        : 'Successfully updated status.';

    setState(() {
      _isSubmitting = false;
```

```
      _errorMessage = null;
      _remarksController.clear();
      _selectedAudit = null;
      _auditRecords = [];
    });
    await showDialog<void>(
      context: context,
      builder: (context) => AlertDialog(
        title: Text(responseMessage),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(),
            child: const Text('OK'),
          ),
        ],
      ),
    );
    _loadAuditIds();
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isSubmitting = false;
      _errorMessage = 'Unable to submit the closure action.';
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Back',
        icon: const Icon(Icons.arrow_back, color: Color(0xFF1A202C)),
        onPressed: () => Navigator.of(context).maybePop(),
      ),
      centerTitle: true,
      title: const Text(
        'Security & Ambiance Closure',
        style: TextStyle(
          fontWeight: FontWeight.w600,
          fontSize: 20,
          color: Color(0xFF1A202C),
        ),
      ),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
    ),
    body: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(
              horizontal: 48.0,
              vertical: 32.0,
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (_errorMessage != null)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 16.0),
                    child: Text(
                      _errorMessage!,
                      style: TextStyle(
                        color: Colors.red.shade700,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                _buildAuditSelectionSection(),
                if (_isLoadingDetails)
                  const Padding(
                    padding: EdgeInsets.symmetric(vertical: 40),
                    child: Center(
                      child: CircularProgressIndicator(
                        color: Color(0xFF005FB8),
                      ),
                    ),
                  ),
                if (_selectedAudit != null && !_isLoadingDetails) ...[
```

```
                    const SizedBox(height: 32),
                    _buildDataTableSection(),
                    const SizedBox(height: 32),
                    _buildClosureActionsSection(),
                    const SizedBox(height: 32),
                  ],
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildAuditSelectionSection() {
    return _SectionContainer(
      title: 'Audit Selection',
      icon: Icons.assignment_ind_outlined,
      child: Row(
        mainAxisAlignment: MainAxisAlignment.start,
        children: [
          _FormDropdown<_AuditOption>(
            label: 'Select Audit ID',
            width: 300,
            value: _selectedAudit,
            items: _auditIds,
            onChanged: _isLoadingList
                ? null
                : (value) {
                    if (value != null) _loadAuditTable(value);
                  },
            displayText: (item) => item.auditId,
            hintText: _isLoadingList ? 'Loading...' : 'Select...',
          ),
        ],
      ),
    );
  }

  Widget _buildDataTableSection() {
    return _SectionContainer(
      title: 'Audit Details Data',
      icon: Icons.table_chart_outlined,
      padding: EdgeInsets.zero,
      child: Padding(
        padding: const EdgeInsets.all(8.0),
        child: ClipRRect(
          borderRadius: const BorderRadius.only(
            bottomLeft: Radius.circular(12),
            bottomRight: Radius.circular(12),
          ),
          child: SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            child: DataTable(
              decoration: BoxDecoration(
                border: Border.all(color: const Color(0xFFEDF2F7)),
              ),
              headingRowColor:
                  MaterialStateProperty.all(const Color(0xFFF9FAFB)),
              dataRowColor: MaterialStateProperty.all(Colors.white),
              dividerThickness: 1,
              horizontalMargin: 24,
              columns: const [
                DataColumn(
                    label: Text('Category',
                        style: TextStyle(
                            fontWeight: FontWeight.w600,
                            fontSize: 12,
                            color: Color(0xFF4A5568)))),
                DataColumn(
                    label: Text('Question',
                        style: TextStyle(
                            fontWeight: FontWeight.w600,
                            fontSize: 12,
                            color: Color(0xFF4A5568)))),
                DataColumn(
                    label: Text('Auditor Remark',
                        style: TextStyle(
                            fontWeight: FontWeight.w600,
                            fontSize: 12,
                            color: Color(0xFF4A5568)))),
                DataColumn(
                    label: Text('BH Answer',
```

```
                    style: TextStyle(
                        fontWeight: FontWeight.w600,
                        fontSize: 12,
                        color: Color(0xFF4A5568)))),
            ],
            rows: _auditRecords.map((record) => DataRow(cells: [
              DataCell(Text(record.category,
                  style: const TextStyle(
                      fontSize: 12, color: Color(0xFF2D3748)))),
              DataCell(
                SizedBox(
                  width: 350,
                  child: Text(record.question,
                      style: const TextStyle(
                          fontSize: 12, color: Color(0xFF2D3748))),
                ),
              ),
              DataCell(Text(record.auditorRemark,
                  style: const TextStyle(
                      fontSize: 12, color: Color(0xFF2D3748)))),
              DataCell(Text(record.bhAnswer,
                  style: const TextStyle(
                      fontSize: 12, color: Color(0xFF2D3748)))),
            ])).toList(),
          ),
        ),
      ),
    ),
  );
}

Widget _buildClosureActionsSection() {
  return _SectionContainer(
    title: 'Closure Action',
    icon: Icons.fact_check_outlined,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.start,
          children: [
            _FormTextField(
              'Remarks',
              width: 400,
              controller: _remarksController,
              hintText: 'Enter remarks...',
            ),
            const SizedBox(width: 32),
            _FormDropdown<int>(
              label: 'Status',
              width: 300,
              value: selectedStatus,
              items: const [1, 2],
              displayText: (item) =>
                  item == 1 ? 'Satisfactory closed' : 'Wrong Rectification',
              onChanged: (value) {
                if (value != null) {
                  setState(() {
                    selectedStatus = value;
                  });
                }
              },
            ),
          ],
        ),
        const SizedBox(height: 20),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton.icon(
              onPressed: _isSubmitting ? null : () => _submitClosure('0'),
              label: const Text('Confirm'),
              style: ElevatedButton.styleFrom(
                backgroundColor: const Color(0xFF16803C),
                foregroundColor: Colors.white,
                padding:
                    const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(6),
                ),
              ),
            ),
            const SizedBox(width: 16),
            OutlinedButton.icon(
```

```
                onPressed: _isSubmitting ? null : () => context.go('/menuPage'),
                icon: const Icon(Icons.exit_to_app),
                label: const Text('Exit'),
                style: OutlinedButton.styleFrom(
                  foregroundColor: const Color(0xFF2D3748),
                  padding:
                      const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
                  side: const BorderSide(color: Color(0xFFCBD5E0)),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(6),
                  ),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

// --- Reusable UI Components
class _SectionContainer extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final EdgeInsetsGeometry padding;
  const _SectionContainer({
    required this.title,
    required this.icon,
    required this.child,
    this.padding = const EdgeInsets.all(24.0),
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 20,
            offset: const Offset(0, 5),
          ),
        ],
        border: Border.all(color: const Color(0xFFEDF2F7)),
      ),
      child: Column(
        children: [
          const SizedBox(height: 8),
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 16.0,
            ),
            decoration: const BoxDecoration(
              color: Color(0xFFF9FAFB),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(12),
                topRight: Radius.circular(12),
              ),
              border: Border(
                bottom: BorderSide(color: Color(0xFFEDF2F7)),
              ),
            ),
            child: Row(
              children: [
                Icon(
                  icon,
                  size: 22,
                  color: const Color(0xFF4A5568),
                ),
                const SizedBox(width: 12),
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFF2D3748),
                  ),
                ),
              ],
            ),
          ),
```

```
          ),
          Padding(
            padding: padding,
            child: child,
          ),
        ],
      ),
    );
  }
}

class _FormDropdown<T> extends StatelessWidget {
  final String label;
  final double width;
  final T? value;
  final List<T> items;
  final void Function(T?)? onChanged;
  final String Function(T) displayText;
  final String hintText;
  const _FormDropdown({
    Key? key,
    required this.label,
    required this.width,
    required this.value,
    required this.items,
    required this.onChanged,
    required this.displayText,
    this.hintText = 'Select...',
  }) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
          ),
          const SizedBox(height: 8),
          Container(
            height: 36,
            padding: const EdgeInsets.symmetric(horizontal: 10),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(6),
              border: Border.all(color: const Color(0xFFCBD5E0)),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.02),
                  blurRadius: 4,
                  offset: const Offset(0, 2),
                ),
              ],
            ),
            child: DropdownButtonHideUnderline(
              child: DropdownButton<T>(
                hint: Text(
                  hintText,
                  style: const TextStyle(fontSize: 12, color: Colors.grey),
                ),
                value: value,
                isExpanded: true,
                icon:
                    const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
                items: items.map((item) {
                  return DropdownMenuItem<T>(
                    value: item,
                    child: Text(
                      displayText(item),
                      style: const TextStyle(fontSize: 12),
                      overflow: TextOverflow.ellipsis,
                    ),
                  );
                }).toList(),
                onChanged: onChanged,
                style: const TextStyle(
                  fontSize: 12,
                  color: Color(0xFF1A202C),
```

```
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _FormTextField extends StatelessWidget {
  final String label;
  final double width;
  final bool readOnly;
  final String? hintText;
  final TextEditingController? controller;
  const _FormTextField(
    this.label, {
    super.key,
    required this.width,
    this.readOnly = false,
    this.hintText,
    this.controller,
  });
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
          ),
          const SizedBox(height: 8),
          Container(
              height: 36,
              decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(6),
                boxShadow: [
                  BoxShadow(
                    color: Colors.black.withOpacity(0.02),
                    blurRadius: 4,
                    offset: const Offset(0, 2),
                  ),
                ],
              ),
              child: TextFormField(
                controller: controller,
                readOnly: readOnly,
                style: const TextStyle(
                  fontSize: 12,
                  color: Colors.black,
                  fontWeight: FontWeight.w500,
                ),
                decoration: InputDecoration(
                  filled: true,
                  fillColor: readOnly ? const Color(0xFFF7F8FA) : Colors.white,
                  contentPadding: const EdgeInsets.symmetric(horizontal: 10),
                  hintText: (hintText?.isNotEmpty ?? false) ? hintText : '-',
                  hintStyle: TextStyle(
                    fontSize: 12,
                    color: readOnly ? const Color(0xFF2D3748) : Colors.grey,
                  ),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
                  ),
                  enabledBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
                  ),
                  focusedBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(
                      color: Color(0xFF005FB8),
                      width: 1.5,
                    ),
                  ),
```

```
                ),
              )),
          ],
        ),
      );
  }
}

// --- Data Models
class _AuditOption {
  const _AuditOption({required this.auditId});
  final String auditId;

  @override
  bool operator ==(Object other) =>
      other is _AuditOption && other.auditId == auditId;

  @override
  int get hashCode => auditId.hashCode;
}

class _AuditRecord {
  const _AuditRecord({
    required this.category,
    required this.question,
    required this.auditorRemark,
    required this.bhAnswer,
  });

  final String category;
  final String question;
  final String auditorRemark;
  final String bhAnswer;

  factory _AuditRecord.fromJson(Map<String, dynamic> json) => _AuditRecord(
        category: json['CATEGORY']?.toString() ?? '-',
        question: json['QUESTION']?.toString() ?? '-',
        auditorRemark: json['AUDITOR_REMARK']?.toString() ?? '-',
        bhAnswer: json['BH_ANSWER']?.toString() ?? '-',
      );
}
```